

به نام خدا

فاز اول پروژه‌ی درس طراحی کامپایلر

پاییز ۱۴۰۴

ددلاین: انتهای جمعه، ۱۴ آذر

طراحان: علی عطاءاللهی، سروش اصفهانیان، مرضیه موسوی

فهرست مطالب

ساخت AST 2
تعریف گره‌های AST 2
ایجاد گره‌ها در گرامر ANTLR 3
نحوه ارزیابی 4
برای :for 4
برای :repeat 4
نودهای خروجی 5
نکات مهم 5

ساخت AST

در این بخش لازم است درخت AST را برای کد ورودی تشکیل دهید. برای این کار لازم است موارد زیر پیاده‌سازی شوند:

- تکمیل گرامر: اضافه کردن کد antlr برای دستور expand، منطبق با داکیومنت زبان
- گره‌های درخت (شامل کلاس‌ها، روابط بین آنها و ارث‌بری)
- ایجاد گره‌ها در گرامر antlr
- اضافه کردن متدهای مربوط به بخش ارزیابی

در این پروژه، لازم است درخت (Abstract Syntax Tree) AST برای کد ورودی تولید شود. AST یک نمایش درختی از کد منبع است که ساختار نحوی آن را به‌صورت سلسله‌مراتبی نمایش می‌دهد و نقش مهمی در پردازش و تجزیه کد دارد. برای پیاده‌سازی AST، مراحل زیر باید انجام شوند:

در این فاز، پس از تکمیل گرامر، لازم است درخت AST برای کد ورودی تولید شود. در کلاس درباره‌ی چگونگی ساخت AST توضیح داده شده است. توجه کنید که طراحی و ساختار درست AST بسیار مهم است زیرا در تمامی فازهای بعدی نیاز است در این درخت پیمایش کنید و از آن استفاده کنید.

نکته: برای پیاده‌سازی AST تنها مجاز به اضافه کردن Semantic Action-ها در میان قوانین گرامر ANTLR هستید و هیچ روش دیگری، مانند Listener، حتی با وجود خروجی درست هم قابل پذیرش نیست.

توصیه می‌شود برای درک بهتر ساختار آن و نحوه پیاده‌سازی آن به نمونه کد SimpleLang در ایلرن مراجعه کنید. همچنین برای پیاده‌سازی آن می‌توانید گام های زیر را طی کنید:

تعریف گره‌های AST

هر عنصر زبانی (مانند دستورات، عبارات، متغیرها و عملگرها) باید به‌عنوان یک گره (Node) در AST نمایش داده شوند. این گره‌ها باید به‌صورت کلاس‌های مجزا تعریف شوند که روابط بین آن‌ها نیز مشخص باشد.

- **ساختار کلاس‌ها**
  - یک کلاس پایه (مثلا Node) برای تمامی گره‌های AST تعریف شود که گره‌های مخصوص به هر بخش از زبان از این کلاس پایه ارث‌بری کنند (مانند Expression و Statement).
  - گره‌های درخت تعریف شود. این گره‌ها می‌توانند چندین مرحله ارث‌بری داشته باشند. به عنوان مثال گره BinaryExpression از گره Expression ارث‌بری می‌کند و گره Expression از Node ارث‌بری می‌کند (ممکن است بتوان نحوه‌ی ارث‌بری کلاس‌ها را به انواع دیگری نیز تعریف کرد). همچنین توجه کنید که در زبان جاوا، یک کلاس نمی‌تواند از بیش از یک کلاس به طور همزمان ارث‌بری کند.
  - هر گره شامل اطلاعات مربوط به خود (مثلا نوع عملیات، مقدار متغیرها یا فرزندان) است.
- **روابط بین گره‌های AST**
  - هر گره‌ای که شامل زیرگره‌ها (فرزندان) است، باید فیلدهای مشخصی برای ذخیره‌سازی آنها داشته باشد. به‌عنوان مثال، گره‌های مربوط به عملگرهای دوتایی (باینری) باید دو فیلد به نام‌های فرزند چپ و فرزند راست داشته باشند که عملوندهای مربوطه را در خود نگه‌دارند (در این حالت در زبان جاوا، این فیلدها به صورت متغیرهایی از نوع رفرنس هستند و آدرس instance-هایی از کلاس‌های فرزندان در آن‌ها ذخیره می‌شود).

ایجاد گره‌ها در گرامر ANTLR

در این مرحله لازم است در کد ANTLR، کدهای جاوا را جهت ساخته شدن درخت AST اضافه کنید. برای این کار مانند کد simpleLang و کارگاه عمل کنید. در هر قانون گرامر لازم است گره مربوط به آن را بسازید و اطلاعات آن را پر کنید. این اطلاعات می‌توانند به طور مستقیم از توکن‌ها گرفته شوند یا اینکه سایر گره‌های AST باشند که زیر درخت آن هستند. در نهایت گره ساخته شده را بر می‌گردانید. لازم به ذکر است که در بعضی قوانین گرامر تنها کافی است گره زیر درخت را برگردانید.

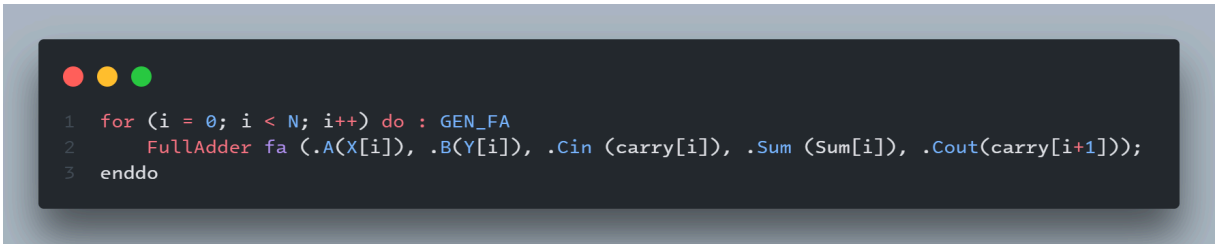
نحوه ارزیابی

برای این که بتوان درخت AST ساخته شده را ارزیابی کرد لازم است موارد زیر را پیاده‌سازی کنید.

موارد زیر را چاپ کنید:

- تعداد statement-ها در گره حلقه for
- تعداد repeat-های موجود در یک configuration یا componenttype

برای for:



```
for (i = 0; i < N; i++) do : GEN_FA
    FullAdder fa (.A(X[i]), .B(Y[i]), .Cin (carry[i]), .Sum (Sum[i]), .Cout(carry[i+1]));
enddo
```

مقدار ۱ باید چاپ شود که یعنی یک فرزند دارد که گزاره‌ی مربوط به ساخت FullAdder است.

توجه کنید که statement-های درون پرانتز که شرایط حلقه را مشخص می‌کنند در نظر گرفته نمی‌شوند و تنها بدنه‌ی for مد نظر است.

برای شمارش تعداد statement-ها توجه کنید که برای حلقه for عبارت های درون پرانتز که شرایط را مشخص می‌کنند، شمارش نمی‌شوند.

برای repeat:

برای مثال زیر، باید مقدار ۴ چاپ شود.

```
configuration Scenario;
  repeat (FOREVER) period=5  timer0 = ~timer0;
  repeat (FOREVER) period=8  timer1 = ~timer1;

  repeat (FOREVER) period=5  display(timer0_buf);
  repeat (FOREVER) period=8  display(timer1_buf);
endconfiguration
```

۴

نودهای خروجی

پیمایش AST :

بعد از این که ast رو کامل ساختید، باید با استفاده از action-هایی که با زبان جاوا بنویسید و به گرامر خود اضافه می‌کنید، خروجی‌های زیر به همراه شماره خط آن‌ها باید چاپ شوند (ترتیب پیمایش مهم نیست و در آخر خروجی ها بر اساس شماره ی خط مرتب می شوند.)

- با رسیدن به configuration و componenttype خروجی به صورت زیر چاپ می شود:

Line: LINE_NUMBER | Name: NAME | Number of ports: NUMEBR_OF_PORTS | Number of component items: NUMBER_OF_COMPONENT_ITEMS_IN_THE COMPONENT | Number of repeat loops: NUMBER_OF_REPEAT_LOOPS_IN_THE_COMPONENT

- با رسیدن به Component Program Interface Instantiation خروجی به صورت زیر چاپ می‌شود:

Line: LINE_NUMBER | ComponentType: COMPONENT_TYPE | Instances: INSTANCE_MAME | Number of ports: NUMBER_OF_PORTS

- با رسیدن به Loop Generate Construct خروجی به صورت زیر چاپ می شود:

Line: LINE_NUMBER | Name: NAME_OF_LOOP | Number of statements : NUMBER_OF_STATEMENTS_IN_THE_LOOP

نکات مهم

- در صورت کشف هرگونه تقلب، نمره صفر لحاظ می‌شود.
- دقت کنید که خروجی‌ها به صورت خودکار تست می‌شوند؛ پس نحوه چاپ خروجی باید عینا مطابق موارد ذکر شده در تست‌ها باشد. **پیشنهاد می‌شود قبل از پیاده‌سازی، samples را بررسی کنید.**
- توجه کنید که علاوه بر تست های قرار داده شده، تعدادی تست دیگر نیز بعد از پایان مهلت ارسال تکلیف قرار می گیرد و نمره نهایی ترکیبی از نمره ی دو نوع تست است.